ZCrypt

A Post-Quantum Shielded Ledger
with Whole-Pool Anonymity

Echelon Labs LLC

Christian Papathanasiou (founder@zcrypt.network)

July 2026

Abstract

We present ZCrypt, a proof-of-stake ledger in which every unit of value lives inside a single shielded pool: there are no transparent transfers. The one value-moving transaction publishes only nullifiers, fresh coin commitments, encrypted notes, and a zero-knowledge proof. The proof is a STARK built entirely from hash functions, with no trusted setup, so the soundness of the money rests on assumptions a quantum computer does not break. Each spend proves membership in the global commitment tree by an in-circuit Merkle path, making the anonymity set the entire pool rather than a sampled decoy ring, and one-input spends are padded with value-zero phantom coins so that arity is invisible. Spending authority is a hash preimage proven inside the circuit (the value layer contains no signatures), and notes to recipients are sealed with ML-KEM-768 against harvest-now-decrypt-later attacks. Every private rule enforced by the circuit has a public twin: a seven-instruction bytecode program, differentially tested against the circuit, that serves as the authoritative specification of a valid transfer. Consensus is Byzantine fault tolerant with instant finality; issuance follows a fixed 21-million schedule audited against a committed state root; the pool carries arbitrary registered assets with hidden type identifiers, issued under rotatable M-of-N ML-DSA threshold authorities; and bitcoin enters and leaves the pool through trustless SPV-verified atomic swaps. The fee for moving value is fixed, paid in the native coin, and burned.

1 Introduction

Public blockchains solved double-spending by publishing everything [1]. The cost of that choice compounds: every payment on a transparent ledger is a permanent, globally replicated record of who paid whom, how much, and when, available to any adversary with a block explorer. The secondary cost is fungibility. Coins with visible histories acquire reputations; exchanges and processors refuse coins "tainted" by prior owners the current holder never met. Cash has neither problem, because a banknote carries no ledger.

Privacy systems built since then each concede something structural. Decoy-based designs [2] hide a spend among a small sampled ring, and the effective anonymity degrades under intersection analysis as rings overlap. Shielded-pool designs [3] achieve strong hiding with elliptic-curve SNARKs; later proof systems removed the original trusted setup [4], and multi-asset shielded pools with in-pool exchange now exist [5, 6], but all of these rest on discrete-logarithm assumptions that quantum computers are expected to break: an uncomfortable foundation for money intended to outlive its designers' hardware assumptions. And nearly all of them keep a transparent ledger alongside the private one, so most value never enters the shielded pool and the pool's anonymity suffers for it.

ZCrypt is built from four commitments:

1. **One pool, one path.** There is no transparent value transfer. Every coin, native or asset, exists only as a hiding commitment in one global pool, and the single spend transaction is the only way value moves. Privacy is not an option that fragments the anonymity set; it is the substrate.
2. **Whole-pool anonymity.** A spend proves, inside the circuit, a Merkle path from the spent coin's commitment to a recent root of the global commitment tree. The anonymity set of every spend is every coin ever created, not a ring of sampled decoys.
3. **Post-quantum by construction.** The proof system is a hash-based STARK with no trusted setup. Spend authority is a hash preimage, not a signature. Note encryption uses a lattice KEM. No discrete-logarithm or pairing assumption protects any unit of value.
4. **Transparent rules over private state.** What the ledger hides is who and how much; what it must never hide is the rules. Every consensus rule, the transfer circuit included, has an executable public specification in a small bytecode VM, differentially tested against the circuit, and total issuance is a public function of block height that anyone can audit against the state root.

The remainder of this paper describes the system as implemented: the cryptographic foundations (§3), coins and addresses (§4), the shielded transfer (§5), the auditability VM (§6), consensus and state (§7), the monetary and staking system (§8), multi-asset support and issuance authority (§9), Bitcoin interoperability and in-pool swaps (§10, §11), the wallet (§12), the privacy and security posture (§13, §14), and deployment models (§15).

2 System Overview

The network is a proof-of-stake replicated state machine with instant finality (§7). Its committed state is small and fully public in structure:

- an append-only Merkle tree of *coin commitments*: one leaf per coin ever created;
- a sparse Merkle tree of *nullifiers*: one entry per coin ever spent;
- the staking ledger, the asset registry, and an escrow book for cross-chain swaps.

A hash of these roots, together with the Bitcoin SPV header-chain digest and the immature-mint queue introduced with the bridge (§10), is the application state root agreed by consensus at every block: seven components in all.

Wallets do the heavy lifting. To pay, a wallet selects its own coins, builds a zero-knowledge proof locally, and broadcasts a transaction containing only the proof and

its public inputs. Validators verify the proof, check the nullifiers are fresh, append the new commitments, and never learn amounts, asset types, or participants. To receive, a wallet trial-decrypts the encrypted notes attached to on-chain transactions; a note that authenticates under the wallet's decapsulation key is a payment to it.

Everything that moves value (payments, gas, stake bonds, bridge deposits and redemptions, staking-reward claims) ultimately resolves to coins entering or leaving this one pool.

3 Cryptographic Foundations

3.1 Field and Hash: Poseidon2 over BabyBear

All in-circuit objects (commitments, nullifiers, public keys, Merkle nodes) are computed with a single algebraic hash: the Poseidon2 permutation [7] over the BabyBear prime field, $p = 15 \cdot 2^{27} + 1 = 2013265921$. The permutation has width 16 (sixteen field elements of state), an x^7 S-box, 8 full rounds and 13 partial rounds. Hashing is by truncated permutation: a fixed 16-element input layout is permuted once and the first 8 output elements are the digest. Because inputs occupy fixed positions in the state (value limbs, blinding limbs, key limbs, asset identifier each have a reserved slot), domain separation between commitment, nullifier, key derivation, and tree compression is positional and unambiguous.

An algebraic hash matters because the same function must be cheap both natively and inside the proof: one Poseidon2 permutation costs 298 trace columns in the circuit, versus tens of thousands for a bit-oriented hash like SHA-256. Preimage resistance of Poseidon2 is what makes spend authority (§4.4) and the burn address (§5.7) meaningful; against a quantum adversary Grover's algorithm gives at most a square-root speedup, which the parameter sizes absorb.

Outside the circuit, byte-level hashing (state roots, transaction binding, key derivation tags, escrow identifiers) uses SHA3-256 [8] with an 8-byte ASCII domain tag and length-prefixed fields, and the FRI commitment layer inside the proof system uses Keccak. Nothing anywhere in the value layer depends on the hardness of discrete logarithms.

3.2 The Proof System: a Hiding STARK

Transfer proofs are STARKs [9] in the AIR/FRI paradigm [10], produced and verified with the Plonky3 toolkit [11]. The arithmetization is a two-dimensional execution trace over BabyBear; constraints are enforced at every row; low-degree testing uses FRI with a blowup factor of 4, folding arity up to 4, and a final polynomial of length 2. Fiat–Shamir challenges are drawn from a degree-5 extension of BabyBear ($\approx 2^{155}$ elements), keeping soundness comfortably above the query bound despite the small base field.

The standard parameterization uses 56 FRI queries plus 16 bits of proof-of-work grinding on both the commitment and query phases, for a conjectured soundness level of 128 bits; a high-security build raises this to 112 queries and 20-bit grinding (≈ 244 bits). The prover asserts at configuration time that conjectured soundness is at least 100 bits.

Zero-knowledge is achieved in the polynomial commitment layer: the Merkle commitments over trace and FRI codewords are *hiding* (each leaf is masked with randomness drawn from a ChaCha20-based CSPRNG seeded from the operating system), and four random codewords blind the quotient. Two proofs of the same statement are byte-wise independent. This hiding layer is a deliberate departure from a stock transparent STARK, whose Merkle openings expose raw trace values: here every opening the verifier sees is masked and the low-degree test is blinded, so the privacy claims of §13 rest on this layer, and a formal simulation-based zero-knowledge analysis of it is the principal piece of outstanding theory work. Crucially, there is *no trusted setup*: the only preprocessing is a deterministic public matrix of selector columns that prover and verifier each derive independently from the circuit description.

3.3 Note Encryption: ML-KEM-768

A payment must convey the coin's opening (value, blinding factor, asset identifier) to the recipient and no one else. Each output slot of a transaction carries an encrypted *note* produced as follows: the sender encapsulates against the recipient's ML-KEM-768 [12] public key; the shared secret is expanded with HKDF-SHA256 [13], with the KEM ciphertext and the recipient's encapsulation key bound into the derivation transcript; the resulting key encrypts the note with ChaCha20-Poly1305 [14]. The plaintext is 48 bytes (value, blind, asset identifier) padded to a fixed 256 bytes, so every note ciphertext on chain has identical size regardless of content.

Recipients scan by trial decryption: a note either authenticates under the wallet's decapsulation key or it is noise. Because ML-KEM is a lattice scheme standardized for post-quantum confidentiality, note ciphertexts harvested from the chain today are not retroactively readable by a future quantum computer, a property classical ECDH-based note encryption cannot offer.

Output slots that should not carry a real note (payments delivered out of band, §4.3) carry *cover notes*: correctly-shaped ciphertexts encapsulated to a discarded key. No observer can distinguish a real note from cover traffic.

4 Coins, Keys, and Addresses

4.1 The Coin

A coin is a quadruple (v, b, s, a): a 52-bit value v in atomic units, a ∼124-bit blinding factor b (four field

limbs), a ∼248-bit spend secret s (eight field limbs), and a 64-bit asset identifier a (0 for the native coin, ZCRYPT). Only one object derived from it ever appears on chain at creation: the **commitment**

$$C = \mathsf{P2}\big(v_{\mathrm{lo}}, v_{\mathrm{hi}}, b, pk, a\big), \qquad pk = \mathsf{P2}(s),$$

where P2 is the truncated Poseidon2 permutation of §3.1 and the value is split into two 26-bit limbs. The commitment binds the owner's public key, so a coin cannot be created "to nobody" and later claimed, and it hides everything: value, owner, and asset type are all inside the hash.

At spend time a second object appears: the **nullifier**

$$nf = \mathsf{P2}\big(s, v_{\mathrm{lo}}, v_{\mathrm{hi}}, b, a\big).$$

The nullifier is a *canonical function of the coin*: the same coin always yields the same nullifier, and computing it requires the spend secret. Spending the same coin twice therefore produces a collision in the public nullifier set, while an observer cannot connect a nullifier to any commitment.

Values are capped at $2^{52} - 1$ atomic units per coin; with six decimal places this is ∼4.5 billion ZCRYPT per coin, far above the total supply, so the cap never binds in practice.

4.2 Key Hierarchy

An account is derived from a 24-word BIP-39 mnemonic [15] (256-bit entropy). The mnemonic seed is domain-hashed into: the spend secret s (eight field limbs); and a deterministic ML-KEM-768 keypair for note encryption. Both are recoverable from the phrase alone. The spend public key is $pk = \mathsf{P2}(s)$, a hash image rather than a group element.

4.3 Addresses and Payment Instruments

An address serializes a kind tag, the 64-byte spend public key, and the 1184-byte ML-KEM encapsulation key, about 1.2 kB in total, encoded in bech32m [16] with human-readable part zc. (The bech32m implementation lifts the 1023-character length limit; the checksum's formal error-detection guarantee applies within it, with $1 - 2^{-30}$ detection beyond.) Three payment instruments share this machinery:

- **Normal address.** The account's primary address. Incoming payments carry a real encrypted note on chain and are discovered automatically by scanning. Recoverable from the mnemonic.
- **Stealth address.** A structurally identical address backed by an independent key set. Hierarchically derived stealth sub-accounts (seeded from the master seed and an index) are used automatically for *change*, so change outputs are unlinkable to the published address; user-created stealth receive addresses use fresh random seeds that are deliberately *not* derivable from the mnemonic and are invisible to the account's view key. For stealth payments the on-chain slot carries a cover note and the real note travels out of band as a file or string; on the wire, a stealth address is indistinguishable from a normal one.
- **Bearer coin.** A coin whose spend secret is embedded in the token itself: 112 bytes (value, blind, spend secret, asset) encoded in bech32m with prefix zcb. Whoever holds the string owns the value: digital cash with no named recipient. Claiming immediately sweeps the coin into a spend re-keyed to the claimant, killing the previous holder's copy of the secret.

View keys. An account can export its ML-KEM key pair *without* the spend secret. The holder of a view key can decrypt every incoming note to the account's normal address (amounts and asset types) but can neither spend nor derive nullifiers, and therefore cannot see when the funds move on. Stealth and bearer instruments are outside the view key's scope by construction. This gives selective, recipient-controlled disclosure for audit or compliance without surrendering custody.

4.4 No Signatures

The value layer contains no signature scheme. Authorization to spend a coin is knowledge of the spend secret s, and that knowledge is proven inside the circuit: the spent commitment is constrained to open to $pk = \mathsf{P2}(s)$ for the s the prover supplies, and the published nullifier is constrained to be derived from the same s. Transaction malleability is closed off separately: every ciphertext, commitment, fee, and the proof itself are bound into a transaction-binding hash recorded with each nullifier, so a transaction cannot be spliced after broadcast. Removing signatures removes the largest classical attack surface: there is no ECDSA or EdDSA anywhere between a user's funds and an adversary.

One consequence deserves stating plainly: there is no in-protocol multi-signature custody of user funds. Because authority is knowledge of s, threshold custody means splitting s (or the mnemonic that derives it) by secret sharing outside the protocol and reconstructing it to spend. Issuance authorities are threshold-governed on chain (§9.1); individual coins are not, and institutional custody of pool coins is today an off-chain key-management problem.

5 The Shielded Transfer

5.1 Transaction Contents

The single value-moving transaction publishes exactly:

- up to 2 **nullifiers** (one per input slot);
- 2 **output commitments** (fresh coins);
- 2 fixed-size **encrypted notes** (real or cover);

- a **fee** field;
- an **anchor root**: a recent root of the commitment tree;
- the **STARK proof**.

Hidden: amounts, asset type, sender, recipient, which coins were spent, and even *how many* coins were spent (§5.6). Public: that some valid spend of pool coins occurred, and the fee field (which is a constant for every user transfer, §5.7).

5.2 The Proven Statement

The proof attests, with respect to 50 public field elements (output commitments, fee, nullifiers, anchor root, a chain-domain tag, and an eight-limb digest binding the transaction's encrypted notes into the proof), that the prover knows openings (v_i, b_i, s_i, a) for each input and (v'_j, b'_j, pk'_j, a) for each output such that:

1. **Membership.** Each active input's commitment is a leaf of the depth-32 commitment tree with the public anchor root, via an in-circuit Merkle path of Poseidon2 compressions. The path proves both existence and position without revealing either.
2. **Ownership.** Each input commitment opens to $pk_i = \mathsf{P2}(s_i)$: the prover knows the spend secret.
3. **Nullifier correctness.** Each published nullifier equals $\mathsf{P2}(s_i, v_i, b_i, a)$ for the same witness values; nullifiers cannot be forged or detached from the coin they cancel.
4. **Output correctness.** Each public output commitment opens to its witness values with a well-formed layout.
5. **Range.** Every value is decomposed into 52 constrained bits; the fee into 26. Nothing overflows the field.
6. **Conservation.** $\sum_i v_i = \sum_j v'_j + fee$, enforced limb-wise with an explicitly range-checked carry, so it holds over the integers.
7. **Single asset.** All inputs and outputs carry one asset identifier. Value cannot leak across asset types inside a transaction.
8. **Domain.** A hash of the chain identifier is a public input, so proofs are not replayable across networks.

The whole-pool membership proof is the heart of the privacy model: the anonymity set of a spend is all 2^{32} leaf slots of the tree (every coin ever created), and it grows with the system. Nothing is sampled, so there is nothing for intersection analysis to intersect.

5.3 Anchors and the Root Window

The commitment tree is append-only, so its root changes with every block that adds coins. A proof is built against the root the wallet last synced; consensus accepts any anchor within a sliding window of the 256 most recent distinct roots, checked *before* proof verification. Wallets therefore never need to race the chain, while proofs against ancient state expire naturally.

5.4 Tree Capacity and Generation Rollover

The membership circuit fixes the Merkle depth, so one commitment tree holds a bounded number of leaves. The reference configuration is depth 32: about 4.29 billion slots. Only real output commitments consume leaves (a transfer appends two for the payment itself and two for its gas note; the value-zero phantom used for arity hiding is an input and is exempt from the tree), so at roughly four leaves per transfer a depth-32 tree holds on the order of a billion transfers before it fills, beyond any near-term horizon. Capacity is nonetheless a deployment and lifetime concern rather than a fixed ceiling, and the protocol addresses it at two levels.

First, depth is a launch parameter. Each level costs one Poseidon2 compression per input on the authentication path, so depth trades directly against proof cost: raising the tree from depth 20 to depth 32, a 4096-fold capacity increase, measures at about 2.4% more proof size and roughly 30% more proving time, with verification unchanged (§6.2). A deployment sizes the tree to its intended lifetime on that curve.

Second, when a tree does fill, the ledger rolls over to a fresh tree of the same depth, a new *generation*. A spend proves membership against the root of its coin's generation, which the public anchor identifies, and the root window spans the boundary so no wallet stalls across a rollover. The nullifier set remains *global*: double-spend prevention is one sparse tree over all generations and is untouched by rollover, so a coin cannot be respent by moving between generations. The single consequence is the anonymity set: within a generation a spend still hides among every coin of that generation (millions of slots), but the set no longer spans all history. Recursive proof aggregation, which folds sealed generations into one accumulator that a spend proves against without an ever-growing path, is the route back to a single whole-history set and is left to a later revision; the generation mechanism is what guarantees the ledger keeps accepting coins in the meantime.

5.5 The Nullifier Set

Spent-coin state is a sparse Merkle tree of height 256 keyed by a hash of the nullifier (limbs reduced to canonical field form first, so no alternative encoding of the same nullifier exists). The leaf *value* is the spending transaction's binding hash, which makes conflicting spends of one coin distinguishable in the tree itself. Insertion is first-spend-wins: a transaction is rejected if any of its nullifiers is present. The tree supports compact inclusion *and non-inclusion* proofs, so a wallet can obtain an authenticated answer to "is this coin spent?" by verifying a proof against the registry root rather than trusting a bare claim (see §12 for how it asks without revealing

which coin it cares about, and the residual root-trust caveat).

5.6 Arity Hiding

A transaction always publishes exactly two nullifiers and two commitments. When a wallet spends a single coin, the second input slot is filled with a deterministic *phantom coin* of value zero: its secret and blind are derived by hashing a domain constant together with the real input's nullifier, it is exempted from tree membership (a value-zero input can create no value), and it publishes a genuine, unpredictable Poseidon2 nullifier like any other input. One-input and two-input spends are therefore indistinguishable on chain, and because the phantom is keyed to the real coin's nullifier, phantoms never collide across transactions. The consensus layer records phantom nullifiers as spent exactly like real ones.

5.7 Gas

Every user transfer pays a fixed fee of 0.01 ZCRYPT, and the fee is *burned*: no validator or treasury receives it. Because the pool hides asset types, the fee cannot be taken as a percentage of a hidden amount; instead a transfer bundles two proofs applied atomically: the *main* transaction (any asset, fee field zero) and a *gas* transaction spending native coins whose first output is required by consensus to be the canonical burn commitment

$$C_{\text{gas}} = \mathsf{P2}(0.01\,\text{ZCRYPT},\, b_{\text{gas}},\, pk_{\text{burn}},\, 0),$$

with the blind b_{gas} revealed in the transaction so the burn is publicly checkable. The burn key pk_{burn} is a fixed constant with no known Poseidon2 preimage: coins committed to it are provably unspendable and permanently out of circulation. ZCRYPT is thus the universal gas asset (transferring any asset consumes native coin), while the transferred asset itself is never taxed and never revealed. The two bundled proofs must not share a coin, and both verify before either applies.

The 0.01 constant is a consensus parameter, changeable only by coordinated upgrade, and it doubles as the spam price: block space is bounded, every occupied slot burns the sender's own ZCRYPT, and a flooder must also bear the full proving cost per transaction, so sustained flooding amounts to a standing donation to every other holder.

6 Transparent Rules: the Auditability VM

A privacy system asks users to trust rules they cannot watch executing. ZCrypt narrows that trust to something inspectable: every transaction type is governed by a short bytecode program for a tiny, purpose-built VM, and that bytecode, not prose, is the specification.

For the shielded transfer, the program is seven opcodes:

CHECK_RANGE · CHECK_MEMBERSHIP · CHECK_SINGLE_ASSET · CHECK_CONSERVATION · COMPUTE_NULLIFIERS · COMPUTE_OUTPUTS · HALT

Each opcode implements, in plain integer arithmetic over the clear witness, exactly one constraint family of the circuit, including reproducing the same phantom-coin padding, bit for bit. The VM cannot run at verification time (the chain never sees witnesses), so it is enforced at the two points where witnesses exist: a differential test suite asserts VM ≡ circuit on valid and adversarial witnesses, and every wallet executes the VM after proving and refuses to broadcast if the VM and the proof disagree, a fail-closed guard against prover bugs.

Transparent transactions (staking, claims, asset registration, authority rotation, escrows) are gated by a second instruction set whose opcodes check stake minimums, claimable balances, ticker validity, escrow state, timeouts, hashlock preimages, burn outputs, and threshold signatures against a registered authority. For these, the VM *is* the on-chain gate: consensus runs the program as the authoritative accept/reject decision. Anyone auditing the chain can read a handful of bytecode programs, a few dozen bytes in total, and know precisely what every transaction type is permitted to do.

6.1 Circuit Engineering

The transfer circuit exists in two differentially-tested forms proving the identical statement with identical public inputs: a *wide* reference trace (32 rows × 22 272 columns, all 72 Poseidon2 permutations side by side) and the production *tall* trace (128 rows × 2 643 columns plus 6 preprocessed selector columns), which runs the 64 Merkle-path compressions one per row instead of side by side. STARK proof size grows linearly with trace width but only logarithmically with height, so the tall layout yields substantially smaller proofs for the same statement; cross-verification between the two forms, and against the VM, is part of the test suite. (The dimensions above are the depth-32 reference configuration of §5.4; the permutation and compression counts scale with tree depth, while the tall column count does not.)

6.2 Measured Performance

The following medians were measured on a consumer laptop, an Apple M3 Max (14 cores, 36 GB), release build, at the standard 128-bit parameterization; 12 proving runs, 40 verification runs, and 2000 VM runs per configuration.

	2-in/2-out	**1-in/2-out**
Prove (build the tx)	107.4 ms	109.3 ms
Verify (network check)	10.4 ms	10.4 ms
Transparent VM audit	73.8 µs	46.4 µs
Proof size	1015 KB	1015 KB
Verify throughput / core	∼96/s	∼96/s

The identical proof sizes are the arity hiding of §5.6 made visible: a single-input spend is byte-for-byte the same shape as a two-input spend. A full user transfer bundles two proofs, the gas note and the main transaction (§5.7), so a wallet builds a complete private payment in about 0.22 s, and a validator core admits it in about 21 ms of proof checking: roughly 48 complete transfers per second per core, or about 670 per second across this one laptop's cores, before consensus overhead.

Set that against the point of sale. A contactless card tap budgets about half a second for the card-terminal exchange and typically completes online authorization in one to two seconds; an inserted chip card takes several. A ZCrypt payment (prove in about 0.22 s, broadcast, and reach BFT finality at the reference deployment's one-second block interval) lands in around a second, with no settlement to follow: the committed block *is* settlement, where a card authorization is only the first step of a clearing process that runs days. In throughput terms, global card networks average on the order of several thousand authorizations per second; at about 670 verified transfers per second per laptop-class chip, matching that raw cryptographic load takes about a dozen such machines. Full privacy and post-quantum soundness turn out to cost less latency than the payment rails in every wallet today.

These figures measure computation, and computation is not the binding resource at scale: bandwidth is. A complete transfer carries roughly 2 MB of proof data, so a network sustaining even tens of transfers per second moves proofs at hundreds of megabytes per minute, and the CPU-bound throughput above will not be reached in practice. Proofs are checked at admission and at finalization but are not part of the committed state (the state root commits to trees and ledgers, not to proof bytes), so deep history can be pruned without weakening it; propagation cost, however, is irreducible in the present design, and the recursive aggregation sketched in §5.4 is the route to compressing it. The card comparison above is therefore a claim about latency and per-payment cryptographic cost, not a claim that a laptop replaces a card network.

7 Consensus and State

7.1 BFT Consensus

ZCrypt is an ABCI application driven by CometBFT [17], a production Byzantine fault-tolerant consensus engine. Blocks are final the moment they commit (no reorganizations, no confirmation heuristics), with sub-second block intervals in the reference deployment. The application validates transactions at mempool admission (full proof verification, anchor and nullifier checks, with no state mutation) and again, authoritatively, at block finalization.

All value flows through the shielded transfer; the remaining transaction types are a closed set of transparent operations: validator bond/unbond, delegation, reward claims, asset registration, and the four escrow operations of the bridge. Any other payload is rejected: there is no script layer and no general-purpose execution to widen the attack surface.

7.2 The State Root

Every block commits an application hash

$$H\big(root_{\mathrm{nf}},\ root_{\mathrm{tree}},\ root_{\mathrm{stake}},\ root_{\mathrm{assets}},\ root_{\mathrm{escrow}},\ root_{\mathrm{btc}},\ root_{\mathrm{immature}}\big)$$

over the nullifier set, the commitment tree, the complete staking ledger (stakes, delegations, rewards, unbonding queue, tombstones, validator keys), the asset registry (every ticker's supply counter and cap), the escrow book, the Bitcoin SPV header chain (§10), and the queue of immature bridge mints. Total issuance is part of the staking digest, so the claim "supply equals the emission schedule" is checkable by anyone against a single committed hash. Full node state serializes to a compact snapshot written atomically at every commit; a node restarted from its snapshot reproduces the committed state root exactly, and a corrupt snapshot halts the node rather than silently rejoining from genesis.

7.3 Dynamic Validator Set

The validator set is not configured; it is a pure function of the staking ledger, with one disclosed bootstrap exception described below. Each block, the application diffs the ledger's validator set before and after transaction application and pushes the resulting power changes to the consensus engine: a successful bond adds a block producer at a voting power of one per whole staked coin, an unbond or slash removes one, and delegation changes reweight power, all deterministically from state that is itself under the state root.

7.4 Bootstrap Liveness Guarantor

A Byzantine fault-tolerant chain cannot finalize blocks until an honest supermajority of stake exists, which a network has not yet acquired at genesis. To bridge that gap, the genesis app_state may designate one foundation validator as a *sentinel*: for as long as it is active, the application holds its voting power at max(floor, $2 \cdot$ others$+1$), just above two-thirds of the total, so the chain keeps producing final blocks even before an independent validator set exists. This is a deliberate, on-chain-visible centralization of the bootstrap phase: while the sentinel is active, that one validator's power is *not* a function of its staked coins, and the network is not yet decentralized. Both the sentinel's identity and its inflated power are committed in the state root, so the arrangement is public rather than hidden.

It is designed to dissolve rather than persist. The sentinel *auto-retires* to its own ordinary staked power once external stake crosses a genesis-set threshold, spread

across at least four distinct independent validators and sustained for a dwell window of blocks (so a transient stake spike by a single well-capitalized party cannot trip it), and it can additionally be retired early by a transaction signed by its own consensus key. After retirement the validator set is purely stake-weighted, with no privileged node. The mechanism is a temporary crutch with a committed, automatic end condition, not a standing authority; operators sizing a deployment choose the retirement threshold as a launch parameter.

8 Monetary System and Staking

8.1 Emission

Supply is fixed at 21,000,000 ZCRYPT with six decimal places, at a one-second target block time. Twenty percent is allocated to the foundation at genesis, but not made liquid: the allocation vests on an on-chain schedule that is a pure function of block height (50,000 ZCRYPT liquid at genesis for validator bonding and operations, a 25% cliff at month twelve, then 87,500 ZCRYPT per month through month forty-eight), and the consensus rules reject any foundation claim ahead of that schedule. The remaining 80% (16.8M) is emitted to validators and delegators on a Bitcoin-style schedule: 0.14 ZCRYPT per block, halving every 60,000,000 blocks (roughly 1.9 years), reaching zero after 63 halvings. Rewards are paid every 100-block epoch, and the cumulative emission is clamped so that issuance can never exceed the schedule; integer rounding dust is simply never minted. There is no mint authority in the code path: issuance is the vested foundation amount plus epoch rewards, both folded into the state root.

The fixed 0.01 ZCRYPT gas burn (§5.7) and slashing burns are permanent supply destruction, so net supply asymptotically approaches a value strictly below 21M.

8.2 Bonding as Provable Burning

A validator joins by *spending shielded coins into the burn key*. The bond transaction carries an ed25519-signed header (the consensus key that will sign blocks, the stake amount, a reward address, the validator's commission, and a per-key monotonic nonce, all signed together with the chain identifier so the authorization is single-use and not replayable across chains or after a re-bond) together with a shielded spend whose first output is required to be the revealed-value commitment of exactly the stake, bound to pk_{burn}. The stake thus provably leaves the pool. The amount is public, as a bond must be, but its history is not: the bonded coins are anonymous pool coins like any others. Minimum self-bond is 10,000 ZCRYPT. Delegation works identically, at any positive amount, to any bonded validator.

8.3 Rewards, Unbonding, Slashing

Epoch rewards are split by weight $w_v = (\text{stake}_v + \text{delegated}_v) \times \text{blocks signed}_v$; an absent validator earns nothing regardless of stake. Each validator sets its own commission on the delegated share, chosen at bond time in basis points (defaulting to 10%), so validators compete on it; the rest is divided pro rata. Rewards accrue to a ledger keyed by reward address and are *claimed* by a gasless transaction that mints coinbase coins directly into the shielded pool, at commitments deterministically derivable by the claimant's wallet, so even freshly-minted rewards enter circulation as ordinary anonymous coins.

Unbonding removes the validator (and cascades its delegations) into a queue released after 10 days of block time; returned stake reappears as claimable balance and re-enters the pool through the same claim path. Equivocation or light-client-attack evidence from the consensus engine burns 5% of the offender's stake and all its delegations, force-unbonds the remainder through the queue, and permanently tombstones the validator's identity. Delegators share slash risk and cannot front-run it: undelegation passes through the same 10-day queue.

9 A Multi-Asset Shielded Pool

The pool is asset-generic. A 64-bit asset identifier lives *inside* each commitment; the circuit enforces that a transaction is single-asset and conserves value, but the identifier never appears on the wire. An observer of a transfer cannot tell ZCRYPT from any other asset: transfers of every asset share one anonymity set.

Assets are created in a first-come registry: a ticker (2–10 alphanumerics, case-insensitive) maps to an asset identifier by hash, with a declared decimal precision and an *immutable* maximum supply. Minting happens only through the escrow mechanism of §10, authorized either by the asset's registered threshold authority (§9.1) or, for zBTC, by an in-consensus Bitcoin SPV proof (§10), and is consensus-capped at the registered maximum supply; the per-asset supply counter is part of the state root. Redemption burns tracked supply. The native asset is identifier 0 and is never registered. The zBTC ticker is special-cased at the registration chokepoint: consensus rejects any registration of it that does not carry exactly eight decimals, the 21-million cap, and the SPV mint form of §10, so both the bridge asset's parameters and its trustless issuance are protocol-pinned rather than registrant-chosen.

9.1 Threshold Mint Authority

An asset's mint authority is an M-of-N threshold of ML-DSA-65 keys [18], the lattice signature standard and sibling to the ML-KEM encryption of §3.3, so issuance authority is post-quantum end to end. The registry stores only a 32-byte *commitment*: a hash of the threshold and the sorted key set (up to 32 keys). To open a mint escrow, the submitter presents the key set and at least M signatures over a message binding the *exact* escrow parameters (asset, value, hashlock, timeout, and recipient key) under a protocol-specific context string. The con-

sensus VM checks that the presented key set hashes to the registered commitment and that the threshold verifies; a signature therefore authorizes one precise issuance and nothing else, and replaying the same authorization collides with the deterministic escrow identifier.

Authorities rotate on chain. Each asset carries a monotonic *authority epoch*, folded into the state root. A rotation transaction presents the *current* key set with M signatures over (asset, current epoch, new commitment); on acceptance the registry swaps the commitment and increments the epoch. Epoch binding makes every rotation signature single-use: even a rotation back to a previous key set cannot replay an old authorization. Signers never reveal secrets on chain in either operation; only public keys and signatures appear.

A single user issuing a personal asset simply uses a 1-of-1 authority derived from their wallet seed, while a consortium asset uses a genuine multi-party set; the machinery is the same at every scale. The bridge asset zBTC is the deliberate exception: it carries no threshold authority at all, its issuance gated instead by the in-consensus Bitcoin SPV proof of §10, so no set of signers can mint it.

Because gas is always native (§5.7), asset transfers carry a zero fee field in the main proof and burn ZCRYPT in the bundled gas proof; the one exception is the bridge redemption path, where the in-transaction fee is paid in the asset being redeemed.

10 Bitcoin Interoperability

Bitcoin enters the pool as **zBTC**, a registered shielded asset with eight decimals and a 21M cap, minted and burned 1:1 against BTC. Unlike federated or custodial wrapped bitcoin, zBTC is minted *trustlessly*: no treasury, signer set, or price oracle authorizes issuance. Every validator verifies Bitcoin directly, through a simplified-payment-verification (SPV) client embedded in consensus, and mints only against a proof that real BTC was locked. The threshold authority of §9.1 governs user-issued assets; zBTC is governed by the SPV gate described here, and consensus rejects any registration of the zBTC ticker that is not in this SPV form.

10.1 An In-Consensus Bitcoin Light Client

The chain maintains a Bitcoin block-header chain as part of its state, its digest folded into the state root. Anyone may submit batches of 80-byte headers through a permissionless relay transaction; the relayer is untrusted, since each header carries its own proof-of-work. Validators check, deterministically and identically, that each header links to its stored parent, meets the target its bits encode, obeys the difficulty-retargeting schedule (Bitcoin's exact 2015-interval timespan with the fourfold clamp), and post-dates its median-time-past, and they adopt the heaviest-work chain exactly as Bitcoin itself does. A production deployment pins a recent header as a checkpoint, so the chain tracks only headers past it rather than syncing from genesis. Rewriting this header chain would require out-mining the honest Bitcoin network, and a single honest relayer defeats one that withholds.

10.2 The Trustless Mint

zBTC's registered authority is not a key-set commitment but a commitment to a single Bitcoin *reserve* public key. To mint, a party submits an inclusion proof: the raw Bitcoin funding transaction together with a Merkle branch proving it sits in a block on the chain's best header chain. Consensus accepts it only if the block is buried at least a network-dependent minimum depth, the branch reconstructs the block's committed Merkle root, and the funded output pays the pay-to-witness-script-hash of exactly the swap's HTLC: the shared hashlock, the pinned reserve key as claimant, the depositor's refund key, and a safe refund locktime; and the locked satoshis must cover the minted base units 1:1. Binding the deposit to the reserve key is what makes the peg solvent rather than merely atomic. Because only the reserve can ultimately claim the deposited BTC, reserves grow in lockstep with supply, and the obvious attack (locking BTC one can reclaim oneself, minting zBTC, then taking the BTC back) is rejected outright. Each funding outpoint is recorded in a consensus set the first time it mints, so one deposit mints exactly once. No signature authorizes the mint's *existence, amount, or backing*: those are a deterministic consequence of Bitcoin's own proof-of-work. The one signature involved selects the *recipient*: the party opening the mint must present an ECDSA signature by the deposit's Bitcoin refund key over the funding outpoint and the chosen shielded recipient key, so only the depositor (who set that refund key, committed in the SPV-proven script) can direct where the zBTC is minted. This binds recipient to depositor and closes a front-running vector in which a third party, seeing the public deposit proof, opens the mint to themselves; it does not reintroduce any trusted authority over issuance. And because no key set can ever commit to a reserve-key authority, the arrangement is one-way; zBTC can never be rotated to discretionary issuance.

Reserve custody is the honest limit of this trustlessness. Consensus constrains issuance (no zBTC exists without a proof of reserve-bound BTC), but it does not constrain the reserve keyholder: the accumulated BTC is spendable by that key alone, so its compromise or loss leaves the zBTC supply intact but strands redemption. Because consensus only ever checks that deposits pay the pinned public key, that key can itself be held in threshold fashion (multi-party ECDSA among independent operators reconstructing no single point of failure) without any protocol change; how the reserve key is held is a deployment decision, and a deployment should publish it.

10.3 Maturity and Reorg Safety

An SPV proof attests that a deposit was confirmed, but Bitcoin offers only probabilistic finality: a sufficiently deep reorganization could orphan a confirmed block. Since a minted zBTC coin enters the shielded pool and becomes unlinkable, it cannot be clawed back after the fact, so the peg must *prevent* an unbacked coin from ever being spent rather than cure it afterward. zBTC borrows Bitcoin's own coinbase-maturity rule: a newly minted coin is held *immature*, outside the spendable pool and thus outside the anonymity set, until its backing deposit is buried beyond reorganization risk. Each block, consensus sweeps the immature set deterministically: a deposit buried to its maturity depth matures into a spendable coin, and a deposit whose block has been orphaned is handled as below.

The maturity depth is *scaled by value*. A reorganization deep enough to orphan a deposit costs, as a lower bound, its depth times the block reward in forgone honest mining, on top of the majority hashpower it demands, so the attack pays only when the unbacked value exceeds that cost. A flat depth would make a tiny shield wait as long as a large one; instead the required depth grows with the minted value, floored at the mint's minimum confirmations and capped near Bitcoin's hundred-block coinbase maturity. Small transfers mature within roughly an hour, and only large ones wait deep, precisely where an attack is worth mounting.

Orphaning is not treated as an immediate void, because a benign reorganization typically re-mines the same deposit into a different block whereas a genuine double-spend omits it. On orphaning, consensus opens a grace window: within it, a relayer may *re-anchor* the immature mint by submitting a fresh inclusion proof for the same deposit txid on the new best chain, moving its anchor forward so it resumes maturing. Only a deposit that never reappears, a real double-spend, fails to re-anchor, and its immature coin is voided when the grace elapses, burning the supply back. All of this is a pure function of the header chain committed in the state root, so every validator agrees on which coins mature, re-anchor, or void.

10.4 The Atomic Swap

Minting and redemption are two hash-time-locked contracts in the classic atomic-swap form [19], sharing one deliberately untagged SHA-256 hashlock, so a single preimage satisfies both Bitcoin script's OP_SHA256 and ZCrypt's escrow check. On Bitcoin the contract is a standard P2WSH output offering preimage-plus-signature to the claimant, or a signature to the refunder after an absolute OP_CHECKLOCKTIMEVERIFY height. On ZCrypt a consensus escrow locks a public (value, asset) pair under a hashlock and a block-height timeout; escrow settlement coins use deterministic public blinds, so the swap amount is public, as an atomic swap requires, but the settled coins re-enter the private pool.

BTC → zBTC. The depositor picks a ≥16-byte random preimage and funds the Bitcoin HTLC (claimable by the reserve with the preimage, refundable to the depositor after its locktime), waits for the deposit to confirm, relays the intervening headers, and submits the mint proof. Consensus opens the mint escrow and, on the depositor's preimage-revealing claim, mints *immature* zBTC; the reserve reads the revealed preimage from ZCrypt and sweeps the BTC. The zBTC matures once its deposit is deep enough. If the proof is never submitted the depositor refunds on Bitcoin, and if the claim never comes the escrow expires and mints nothing.

zBTC → BTC. Symmetric with roles reversed: the redeemer locks zBTC in a burn escrow, a liquidity provider funds a Bitcoin HTLC claimable by the redeemer, the redeemer's Bitcoin claim reveals the preimage, and the provider uses it to finalize the burn, dropping tracked supply. Every failure path refunds, and at no point does either side hold both legs.

Minting is thus trustless: no party can create zBTC without a proof of real, sufficiently-buried, reserve-bound BTC, and the protocol-pinned cap bounds supply regardless. Redemption is atomic and cannot cheat the redeemer, but it depends on a liquidity provider holding BTC to take the other side, which is a market property rather than a trust assumption in the peg's integrity. The construction removes the federation that federated wrapped-bitcoin designs rely on, at the cost of a maturity delay and a dependence on redemption liquidity.

11 Private In-Pool Swaps

The same escrow machinery lets two parties atomically exchange one shielded asset for another *without either asset leaving the pool*: peer-to-peer trading with no order book, no pooled liquidity, and no counterparty risk.

A third escrow kind, the *swap* escrow, locks value out of the pool the same way a redemption does: a shielded spend whose first output is a revealed-value commitment to the burn key, accompanied by the asset identifier, a hashlock, a timeout, a counterparty key, and a refund key. Claiming with the preimage before timeout re-materializes the locked value as a pool coin for the *counterparty*; expiry re-materializes it for the *locker*. Supply counters are untouched in either case: a swap moves ownership, not issuance, and requires no authority.

Two parties who have agreed a price off chain (a request-for-quote exchange between wallets; the chain never sees or validates the ratio) execute as follows. The taker picks a random preimage and locks their side (say x units of asset A), claimable by the maker. The maker, seeing the escrow's hashlock on chain, locks y units of asset B under the *same* hashlock, claimable by the taker. The taker claims the maker's escrow, revealing the preimage on chain; the maker claims the taker's es-

crow with it. If either side stalls, both escrows expire back to their lockers. It is the Bitcoin swap of §10 with both legs on one chain, and it settles in a handful of blocks.

Privacy composes the same way as the bridge: the swapped amounts and hashlock are public (an atomic swap requires it), but which coins funded each lock is hidden behind the usual whole-pool membership proof, change returns as an ordinary commitment with an encrypted note, and the coins each party receives are standard pool coins whose subsequent spends hide among everything else. The escrow fee is paid in the asset being locked: the pool hides asset types, so in-escrow fees are in-asset, just as on the redemption path.

12 The Wallet

The reference wallet is a desktop application whose backend embeds the full prover; no third party ever sees a witness.

Proving and sending. The wallet maintains an incrementally-synced copy of the commitment-tree leaves, rebuilds the tree locally, and derives Merkle paths for its own coins. A payment selects up to two coins, builds the gas note and the main transaction, proves both (about 0.22 s on a consumer laptop, §6.2), runs the auditability VM as a final cross-check, and broadcasts the bundle. Change always goes to a fresh hierarchical stealth sub-address, and every output slot carries a fixed-size note, real or cover, so delivery mode is not fingerprintable.

Receiving. Scanning fetches only transaction-bearing blocks and trial-decrypts every note against the account key, its stealth change accounts (with a gap limit), and any imported stealth seeds. Stealth payments arrive as an out-of-band note string; bearer coins are claimed and immediately swept to a fresh secret.

Private spent-status checks. Asking a node "is nullifier X spent?" would reveal coin ownership. The wallet never asks about fewer than 32 nullifiers at once: its real queries are shuffled among fresh random decoy values, so the node learns only that someone asked about a batch and cannot tell which entries mattered. Each answer is an authenticated SMT inclusion/non-inclusion proof, which the wallet verifies against the registry root the node reports, deriving the spent/unspent status from the verified proof rather than from the node's bare word; an unverifiable or inconsistent answer is rejected rather than trusted. Removing the last sliver of trust, so the node also cannot answer against a registry root that consensus never committed, requires binding that root to the block application hash through validator-signed headers, a light-client step the reference wallet does not yet perform.

Network privacy and custody. All node RPC can be routed over Tor (on by default, failing closed on misconfiguration). The mnemonic is encrypted at rest under an Argon2id-derived key [20] (64 MiB, 3 passes); the coin database is encrypted under a key derived from the seed. The wallet surfaces the full product: balances per asset, three payment instruments, view-key export, staking-reward claims, asset registration and minting, and both directions of the BTC bridge with automatic recovery of interrupted swaps.

13 Privacy Analysis

What an observer of the chain sees, per action:

- **Transfer:** two nullifiers, two commitments, two uniform ciphertexts, the constant 0.01 ZCRYPT gas burn, an anchor root, a proof. No amounts, no asset type, no participants, no arity, no linkage between the nullifiers and any prior commitments.
- **Reward claim:** amount and recipient shield key (public by design: rewards are public income); the minted coins are anonymous thereafter.
- **Bond / delegate / unbond:** stake amount and validator identity, public by design, with the source coins anonymous.
- **Bridge or in-pool swap:** the swapped amounts and the escrow parameters; the coins funding each leg, and the coins received, are anonymous pool coins.
- **Asset registration or authority rotation:** ticker, decimals, cap, authority commitment and epoch; never a signer's secret.

The residual metadata surface is deliberate and small: everything that *must* be public for consensus to enforce a rule about it (stakes, issuance, escrow amounts, supply caps) is public; everything else is not. Timing and network-origin metadata are mitigated by Tor by default, uniform transaction shape, cover notes, decoy-batched queries, and the absence of any transparent value layer that would otherwise carve the user base into pools of differing privacy.

The anonymity set deserves emphasis: because membership is proven against the whole commitment tree, *every* spend hides among every coin in history, spent or unspent; currently a tree of 2^{32} slots, a parameter of the circuit rather than of the analysis, and extended by generation rollover once full (§5.4), at which point the set spans the current generation rather than all history. There is no decoy-selection distribution to bias and no ring-intersection attack to mount.

14 Security Considerations

Soundness. Value integrity rests on the STARK (conjectured 128-bit soundness in the standard build, ≈244-bit in the high-security build), Poseidon2's preimage and collision resistance, and consensus correctness. All are hash-based; none is threatened by known quantum al-

gorithms beyond Grover-type square-root speedups already absorbed by parameter sizes.

Encoding discipline. Field elements admit no alternative byte encodings anywhere they matter: transaction parsing rejects non-canonical limbs, and nullifier registry keys are computed over reduced forms, eliminating the class of double-spend that respends a coin under a second encoding of the same nullifier.

Replay and binding. Proofs bind a chain-domain tag (no cross-chain replay), an anchor from a bounded recent window (no proofs against ancient state), and a transaction-binding hash covering all ciphertexts and the proof (no post-broadcast splicing). Phantom-coin nullifiers are keyed to the real input's nullifier, so padding can never collide across transactions even under spend-secret reuse.

Consensus economics. Safety violations are punished by evidence-triggered slashing and permanent tombstoning; delegators share the penalty and cannot exit ahead of it. Once the bootstrap sentinel (§7.4) has retired, validator power derives solely from provably-burned pool coins, so acquiring voting power has an on-chain, supply-visible cost; while the sentinel is active, the foundation node holds a disclosed, state-root-committed supermajority that is the explicit trust assumption of the bootstrap phase.

Issuance authority. Mint authorizations are ML-DSA-65 threshold signatures over the complete escrow parameters under a protocol context string, so an authorization fits exactly one issuance, cannot be repurposed as a rotation (rotations carry a distinct in-message tag), and cannot be replayed (the deterministic escrow identifier collides). Rotations are bound to a monotonic per-asset epoch committed in the state root, so no superseded key set retains any power. Malformed authorization bundles (out-of-range or duplicate signer indices, undecodable keys or signatures, trailing bytes) reject the bundle whole.

Quantum posture. The value layer uses no discrete-logarithm assumption: proofs and spend authority are hash-based, and note confidentiality is lattice-based, protecting recorded ciphertexts against harvest-now-decrypt-later. The classical primitives that remain are peripheral and replaceable: ed25519 on validator consensus keys (an availability concern, not a custody concern) and secp256k1 on the Bitcoin leg of swaps, which is a property of Bitcoin, not of ZCrypt.

Assurance. The circuit is differentially tested against an independent wide-trace implementation and against the transparent VM on valid and adversarial witnesses; wallets re-run the VM before broadcast; nodes reproduce their state root from persisted snapshots and refuse to start otherwise. The trusted computing base for "no coin is created except by the rules" is the circuit, the seven-opcode program that mirrors it, and the emission arithmetic: all small, and all public.

Machine-checked verification. The differential tests above compare the circuit to its specification on sampled witnesses; a companion effort raises part of that assurance to machine-checked proofs in the Lean 4 proof assistant [26], kernel-verified with no unproven step (no sorry). Two things are established. First, the transfer's value-integrity specification (the transparent VM of §6) and the arithmetic at the core of the circuit are formalized and proven directly: an accepted transfer creates no value (total outputs never exceed total inputs), and the multi-limb, range-checked carry that reconciles the circuit's field arithmetic with integer arithmetic provably cannot wrap, so conservation in the field is conservation over the integers. Modeling Poseidon2 as collision-resistant, the coin layer follows as theorems: a commitment binds its coin uniquely (value and asset cannot be forged into it), a nullifier binds the coin it cancels (distinct coins yield distinct nullifiers, and a re-spend yields the same nullifier the on-chain set rejects), the commitment fixes the owner key, and commitments, nullifiers, and key images are mutually domain-separated. Second, to connect the proofs to the source rather than to a re-transcription of it, the value-integrity core of the VM is mechanically extracted from its Rust to Lean using Charon and Aeneas [27], the Rust-to-Lean pipeline developed for the Ethereum Foundation's verification of Plonky3, and the no-inflation theorem is re-proved against that machine-generated model.

We state the boundary precisely, because for a proof system the boundary is the point. What is verified is the transfer's *specification* together with its arithmetic and coin-level logic, reduced to standard cryptographic assumptions: the collision-resistance of Poseidon2, the FRI soundness of §3.2, and the lattice assumptions of §3.3. Formal methods reduce a system to such assumptions; they do not discharge them, and every deployed system rests on assumptions of this kind. What is *not* yet machine-checked is the full arithmetic constraint system, the AIR, itself: proving that the deployed constraints enforce exactly the specification and admit no under-constrained witness requires a formal model of AIR and STARK semantics, and is left to future work, with the differential tests the present bridge between specification and circuit. Machine-checked verification also does not substitute for external audit. The proofs use only the Lean core library, depend on no axioms beyond the standard hash assumption at the coin layer, and reproduce from the published artifact with a single build command.

15 Deployment Models

The architecture described in this paper (a shielded pool with hash-based proofs, threshold-governed issuance, selective disclosure, and post-quantum cryptography throughout) is a neutral cryptographic protocol. It can serve as the foundation for private digital cash or as the technical infrastructure for a central bank digital cur-

rency. We describe both possibilities without advocating for either.

15.1 Central Bank Digital Currency

The CBDC design space is defined by a tension: monetary authorities require control over issuance and compliance, while citizens retain a legitimate expectation of transactional privacy, the property physical cash has always provided. As a recent BIS working paper [21] observes, "better technological solutions offering hard privacy need to emerge." The primitives in this paper happen to map onto that tension. The threshold mint authority (§9.1) is an exclusive, consensus-enforced issuance mechanism that distributes signing power across M-of-N institutions and rotates without redeployment. The shielded pool preserves transactional privacy for ordinary commerce. The nullifier registry provides the global double-spend prevention a monetary authority requires. The consensus layer can be operated by the issuer or licensed delegates; the transfer layer remains private.

A CBDC built on these primitives would enable privacy-preserving compliance through tiered disclosure: the architectural inverse of designs that begin with full visibility and attempt to carve out limited privacy. Routine transactions are fully private, equivalent to physical cash. The issuer sees aggregate nullifier volume (a privacy-preserving proxy for monetary velocity) but not individual transactions. Where policy demands more, disclosure is additive and key-scoped: view keys (§4.3) let a wallet disclose its incoming flows (amounts and asset types) to a designated auditor or, under court order, a financial intelligence unit, without surrendering spending authority. Privacy is the base layer; compliance is layered on top of it.

We document the surveillance risk honestly. Privacy infrastructure deployed by states can be repurposed: threshold reporting creates a policy surface that can be lowered to zero; judicial disclosure depends on courts that may operate under political pressure; view-key mandates at issuance would transform a privacy tool into an escrow system. Our architectural position is that the zero-knowledge proof system contains no compliance hooks, trapdoors, or key escrow. Privacy is a property of the cryptography, not a policy setting that can be revoked.

Central banks operate on generational timescales, which makes the post-quantum question acute. A CBDC deployed on elliptic-curve cryptography faces a store-now-decrypt-later threat: capture today, decrypt when quantum computers mature. Signature-based CBDC architectures also hit a quantum-safe performance wall: BIS Project Tourbillon [22] documented roughly a 5× latency penalty and a 200× throughput reduction when moving blind-signature eCash [23] to post-quantum primitives. The proof-based architecture described here avoids this entirely: its value layer is post-quantum from genesis, with no signature ceremony on the payment path at all. None of this constitutes advocacy: whether any of these properties are desirable in a CBDC is a question for monetary authorities and the publics they answer to. We describe what the technology can do, not what it should.

15.2 Private Digital Cash

Deployed as an open network, ZCrypt's instruments behave like cash. A bearer coin is value embodied in a string: whoever holds it holds the funds, and it can be handed over through any channel (a messaging application, an email, a QR code, a USB device) with no intermediary, no account, and no identity verification. A stealth payment travels the same way as an encrypted note, leaving nothing on chain that any view key can open. Finality is a sweep or claim against the network when connectivity allows; until then the token itself is the payment, as a banknote is.

This is the model Chaum envisioned in 1983 [24, 25]: ecash as a privacy-preserving medium of exchange, realised without his reliance on a centralised mint. Remittances settle in seconds over the channels people already use, with no payment processor extracting a fee or recording the transaction. The system also serves organisational uses that require confidentiality: corporate treasuries gain post-quantum protection and privacy from competitors monitoring on-chain movements; funds can move without signalling intent to markets; aid organisations can disburse without creating ledgers that hostile actors use to identify beneficiaries.

16 Conclusion

ZCrypt makes one structural bet: that a ledger is most private, and most fungible, when privacy is not a feature but the substrate. One pool holds all value; one transaction moves it; the anonymity set is the history of the system itself. Around that core, the design insists on a discipline that privacy systems often invert: the *rules* are radically transparent (bytecode specifications, an auditable emission schedule, a state root that commits to every ledger), while the *activity* is hidden. And it refuses to build money on assumptions with a known expiry date: hash-based proofs without ceremony, preimage-based ownership without signatures, lattice-based confidentiality without harvestable ciphertexts. Stake bonding, reward issuance, asset creation, and bitcoin itself all flow into and out of the same shielded pool through the same narrow, provable gates. The result is a system with the auditability of a transparent chain where it counts, and the discretion of cash everywhere else.

Acknowledgments

Thanks to Vasily Pozdyshev for his feedback on institutional adoption, Jameson Lopp for his feedback on non-inclusion proofs, and the zero-knowledge and post-

quantum cryptography communities for advancing the primitives used here.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[2] N. van Saberhagen, "CryptoNote v2.0," 2013.

[3] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox, "Zcash Protocol Specification," 2016.

[4] S. Bowe, J. Grigg, D. Hopwood, "Recursive Proof Composition without a Trusted Setup," IACR ePrint 2019/1021.

[5] H. de Valence et al., "The Penumbra Protocol," Penumbra Labs, 2024.

[6] C. Goes, A. Sun Yin, A. Brink, "Namada: Interchain Asset-Agnostic Privacy," 2022.

[7] L. Grassi, D. Khovratovich, M. Schofnegger, "Poseidon2: A Faster Version of the Poseidon Hash Function," AFRICACRYPT 2023.

[8] National Institute of Standards and Technology, "FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," 2015.

[9] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, "Scalable, Transparent, and Post-Quantum Secure Computational Integrity," IACR ePrint 2018/046.

[10] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, "Fast Reed–Solomon Interactive Oracle Proofs of Proximity," ICALP 2018.

[11] Polygon Zero / Plonky3 contributors, "Plonky3: A Toolkit for Polynomial IOPs," 2024.

[12] National Institute of Standards and Technology, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," 2024.

[13] H. Krawczyk, P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, 2010.

[14] Y. Nir, A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, 2018.

[15] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe, "BIP-39: Mnemonic Code for Generating Deterministic Keys," 2013.

[16] P. Wuille, G. Maxwell, "BIP-350: Bech32m Format for v1+ Witness Addresses," 2020.

[17] E. Buchman, J. Kwon, Z. Milosevic, "The Latest Gossip on BFT Consensus," arXiv:1807.04938; CometBFT, the maintained implementation.

[18] National Institute of Standards and Technology, "FIPS 204: Module-Lattice-Based Digital Signature Standard," 2024.

[19] T. Nolan, "Alt Chains and Atomic Transfers," bitcointalk.org, 2013.

[20] A. Biryukov, D. Dinu, D. Khovratovich, "Argon2: The Memory-Hard Function for Password Hashing and Other Applications," RFC 9106, 2021.

[21] R. Auer et al., "Privacy-Enhancing Technologies for Digital Payments," BIS Working Papers No. 1242, January 2025.

[22] BIS Innovation Hub Swiss Centre, "Project Tourbillon: Exploring Privacy, Security and Scalability for CBDCs," Bank for International Settlements, November 2023.

[23] D. Chaum, T. Moser, "eCash 2.0: Inalienably Private and Quantum-Resistant to Counterfeiting," 2022.

[24] D. Chaum, "Blind Signatures for Untraceable Payments," Advances in Cryptology: CRYPTO '82, 1983.

[25] D. Chaum, A. Fiat, M. Naor, "Untraceable Electronic Cash," Advances in Cryptology: CRYPTO '88, 1990.

[26] L. de Moura, S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," CADE 28, 2021.

[27] S. Ho, J. Protzenko, "Aeneas: Rust Verification by Functional Translation," Proc. ACM Program. Lang. (ICFP), 2022.